Compliance Ambiental Logtech

Pipeline CI/CD, Containerização e Orquestração de Ambientes (Staging e Produção)

Documentação Técnica — Desafio "Navegando pelo mundo DevOps"
Tema ESG: Governança e Compliance Ambiental (Logtech / Comex)

Integrante

Eryk Vieira — RM 564801

Aplicação base: API RESTful em C# / .NET 8 desenvolvida no desafio anterior
Repositório: https://github.com/eryk-vieira/compliance-ambiental-devops
Execução de referência do pipeline: Actions run #2 (commit 47a7b75) — Success em 10m30s

FIAP — Outubro de 2026

1. Visão geral da solução

A aplicação escolhida foi a **Compliance Ambiental Logtech API** (.NET 8), que monitora a pegada de carbono das operações logísticas de transportadoras, valida licenças ambientais e gera alertas de ultrapassagem do teto mensal de CO_2. Para este desafio ela foi adaptada para um fluxo de entrega contínua completo: cada push no GitHub compila a solução, executa os testes automatizados, gera uma imagem Docker imutável, publica-a no GitHub Container Registry e a implanta com Docker Compose em dois ambientes isolados — **staging** e **produção** — sendo o segundo protegido por aprovação manual.

Adaptações feitas na aplicação para suportar o ciclo DevOps:

Adaptação	Motivo
Health checks **/health/live** e **/health/ready** (este último verifica a conexão com o banco via EF Core)	Permitem que o HEALTHCHECK do Docker, o depends_on: service_healthy e o compose up --wait saibam quando a API está realmente pronta.
Endpoint público **/api/info** (ambiente, versão, commit, número do build, data do deploy, host)	Prova objetiva de qual versão está rodando em cada ambiente; usado pelo smoke test para validar o ambiente esperado.
Configuração 100% por variáveis de ambiente (connection string, JWT, APP_ENVIRONMENT)	A mesma imagem roda em dev, staging e produção; nenhum segredo fica dentro da imagem.
3 testes novos (ObservabilityEndpointsTests) e correção na WebApplicationFactory	Cobrem os endpoints novos; a correção removeu uma dependência de API interna do EF Core que impedia a compilação no CI.
Scripts deploy.sh, smoke-test.sh e collect-evidence.sh	Mesma lógica de deploy e validação usada localmente e no pipeline (sem duplicação).

Fluxo resumido:

```
push -> [build .NET 8] -> [test xUnit] -> [docker build & push GHCR (tag = commit)]
     -> [deploy STAGING  : compose :8081 + smoke test + re-tag :staging]
     -> (aprovação manual no environment "production")
     -> [deploy PRODUÇÃO : compose :8082 + smoke test + re-tag :production/:latest]
```

2. Descrição do pipeline CI/CD

Ferramenta: GitHub Actions (workflow .github/workflows/ci-cd.yml). **Registro de imagens:** GitHub Container Registry (ghcr.io). **Ambientes:** GitHub Environments staging e production, o segundo com required reviewers (aprovação manual).

Gatilhos

Evento	Comportamento
pull_request para main	build + testes (quality gate; não gera imagem nem faz deploy)
push em develop	build + testes + imagem Docker + deploy em staging
push em main	tudo acima + deploy em produção após aprovação
workflow_dispatch	execução manual pela interface do GitHub

Etapas (jobs) e lógica

#	Job	O que faz	Saída
1	build	actions/setup-dotnet 8 → cache NuGet → dotnet restore → dotnet build -c Release → dotnet publish. Gera versão 1.0.<run>+<sha>.	artefato api-publish-<sha>
2	test	dotnet test da solução (xUnit + WebApplicationFactory + EF InMemory): 15 testes. Relatório TRX publicado como check no commit (dorny/test-reporter) e cobertura coverlet.	artefato test-results-<sha>
3	docker	docker buildx build multi-stage com cache do GitHub Actions e push para o GHCR com as tags sha-<commit>, <branch> e build-<n>. build-args injetam versão/commit nos labels OCI e no /api/info.	imagem no GHCR
4	deploy-staging	Environment staging. Gera .env.staging a partir de secrets/variables, roda scripts/deploy.sh staging (docker pull → compose up -d --wait → smoke test com 12 verificações), coleta evidências e promove a imagem para a tag :staging.	artefato evidence-staging-<sha>
5	deploy-production	Somente em main e após o sucesso de staging. Environment production aguarda a aprovação do revisor. Mesmo fluxo com docker-compose.production.yml (porta 8082, banco não exposto, limites de memória, rotação de logs). Promove a mesma imagem para :production e :latest.	artefato evidence-production-<sha>

Princípios aplicados

Build once, deploy many: a imagem é construída uma única vez e promovida entre ambientes por re-tag (docker buildx imagetools create) — staging e produção executam exatamente o mesmo binário. **Quality gates encadeados:** nenhuma imagem é gerada se build ou testes falharem e produção só é acionada se o smoke test de staging passar. **Configuração fora da imagem:** segredos vêm dos GitHub Secrets/Variables de cada environment (ou de .env.<ambiente> localmente). **Deploy verificado:** compose up --wait aguarda os healthchecks de SQL Server e API, e o smoke test valida readiness, ambiente reportado em /api/info, 401 sem token, login JWT e 7 endpoints de negócio. **Evidências automáticas:** cada deploy publica compose ps, imagens, redes, volumes, labels, logs e respostas da API como artefatos e no Job Summary. **Ambientes efêmeros no runner:** ao final de cada job o compose é derrubado; o mesmo scripts/deploy.sh funciona sem alteração em um servidor real ou self-hosted runner.

3. Docker: arquitetura, comandos e imagem criada

Arquitetura dos containers

Cada ambiente é um projeto Docker Compose independente composto por dois serviços na mesma rede bridge privada: **db** (SQL Server 2022, imagem oficial mcr.microsoft.com/mssql/server) com volume nomeado para persistência e healthcheck via sqlcmd, e **api** (imagem da aplicação) que só inicia após o banco ficar healthy. As migrações EF Core e o seed são aplicados automaticamente no startup da API.

	docker-compose.yml (dev)	docker-compose.staging.yml	docker-compose.production.yml
Projeto compose	compliance-dev	compliance-staging	compliance-production
Porta da API	8080	8081	8082
Porta do banco no host	1433	14331	não exposta (expose interno)
ASPNETCORE_ENVIRONMENT	Development	Staging	Production
Banco de dados	ComplianceAmbiental	ComplianceAmbiental_Staging	ComplianceAmbiental
Volume	mssql-data	mssql-data-staging	mssql-data-production
Rede	compliance-net	compliance-staging-net	compliance-prod-net
Imagem	build local	ghcr.io/...:sha-<commit>	mesma imagem promovida
Extras	—	—	limite de memória 512M, logs com rotação, restart: always

Dockerfile (multi-stage)

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build          # estágio 1: SDK completo
WORKDIR /src
COPY ComplianceAmbiental.sln ./
COPY src/ComplianceAmbiental.Api/*.csproj src/ComplianceAmbiental.Api/
COPY tests/ComplianceAmbiental.Tests/*.csproj tests/ComplianceAmbiental.Tests/
RUN dotnet restore ComplianceAmbiental.sln                 # camada cacheada
COPY . .
RUN dotnet publish src/ComplianceAmbiental.Api/ComplianceAmbiental.Api.csproj \
    -c Release -o /app/publish --no-restore /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime         # estágio 2: só runtime
ARG APP_VERSION=dev  ARG APP_COMMIT_SHA=local  ARG APP_BUILD_NUMBER=local  ARG BUILD_DATE=unknown
LABEL org.opencontainers.image.version="${APP_VERSION}" org.opencontainers.image.revision="${APP_COMMIT_SHA}" ...
RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080 ASPNETCORE_ENVIRONMENT=Production APP_VERSION=${APP_VERSION} ...
EXPOSE 8080
USER app                                                   # usuário não-root
HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=5 \
    CMD curl -fsS http://localhost:8080/health/ready || exit 1
ENTRYPOINT ["dotnet", "ComplianceAmbiental.Api.dll"]
```

Estratégias: multi-stage (imagem final só com o runtime ASP.NET, sem SDK nem código-fonte); restore em camada separada (rebuilds só baixam pacotes quando os .csproj mudam); .dockerignore excluindo bin/obj, docs, .git e .env; usuário não-root; HEALTHCHECK embutido; metadados OCI via build-args; cache de camadas no CI (type=gha).

Comandos utilizados

```
# local (dev)
cp .env.example .env && docker compose up --build -d && docker compose logs -f api
```

```
# staging / produção (local ou no runner) — mesmo script usado pelo pipeline
bash scripts/deploy.sh staging        # pull/build -> compose up -d --wait -> smoke test
bash scripts/deploy.sh production
bash scripts/smoke-test.sh http://localhost:8082 production
bash scripts/collect-evidence.sh production
# no pipeline (jobs docker / deploy)
docker buildx build --push --build-arg APP_VERSION=... -t ghcr.io/eryk-vieira/compliance-ambiental-api:sha-47a7b75 .
docker compose --env-file .env.staging -f docker-compose.staging.yml up -d --wait --wait-timeout 300 --no-build
docker buildx imagetools create -t ghcr.io/.../compliance-ambiental-api:staging ghcr.io/.../compliance-ambiental-api:sha
```

Imagem criada

ghcr.io/eryk-vieira/compliance-ambiental-api — pública, 2 arquiteturas listadas pelo GHCR, tags da execução de referência: sha-47a7b75 (imutável), main, build-2, staging, production e latest. Labels OCI: version=1.0.2+47a7b75, revision=47a7b75894093eaf652a5a2f44d42d52409de66e. Containers em execução reportados por docker compose ps: compliance-staging-api / compliance-staging-db (healthy, 8081) e compliance-prod-api / compliance-prod-db (healthy, 8082, banco sem porta publicada).

4. Prints do pipeline rodando (build, testes, deploy)

Execução #2 — https://github.com/eryk-vieira/compliance-ambiental-devops/actions/runs/38490906947

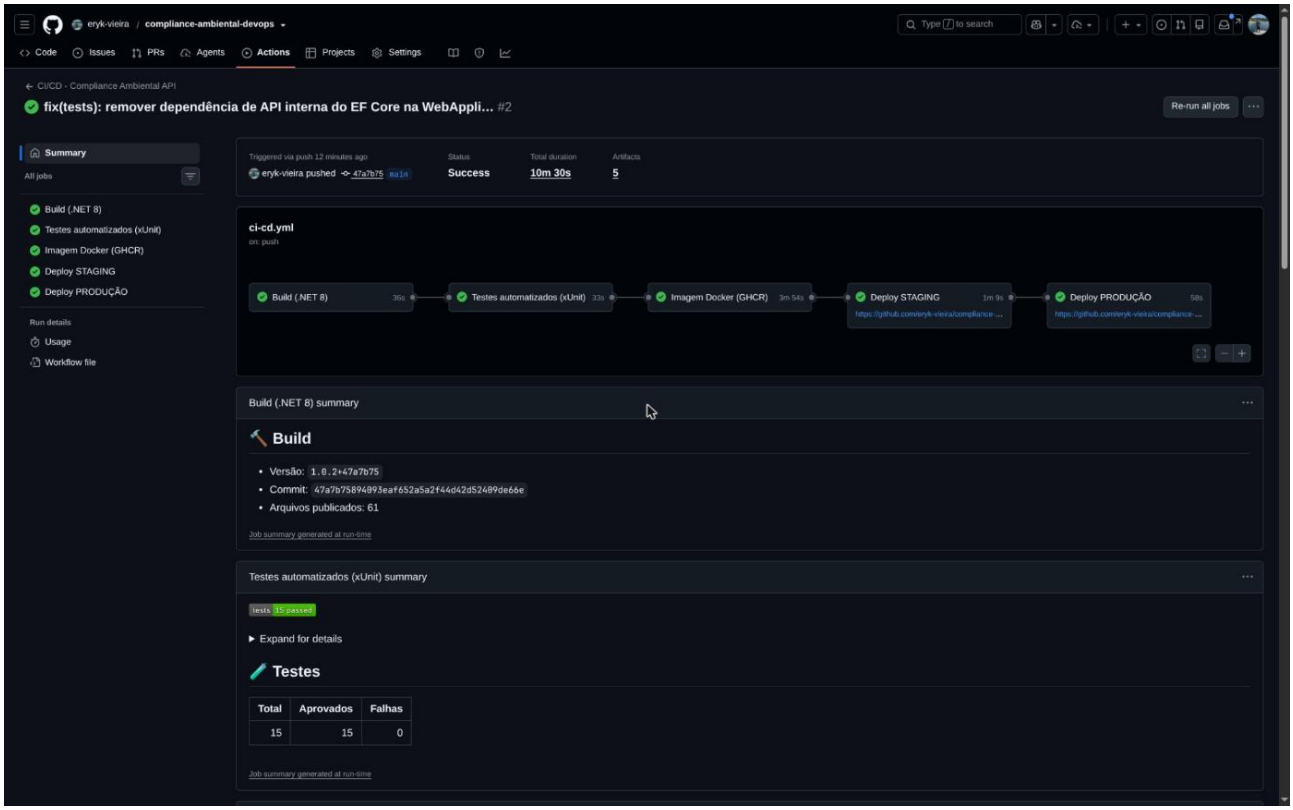


Figura 1 — Workflow completo com os 5 jobs verdes (build 36s, testes 33s, imagem 3m54s, staging 1m09s, produção 58s). Status Success, 10m30s, 5 artefatos.

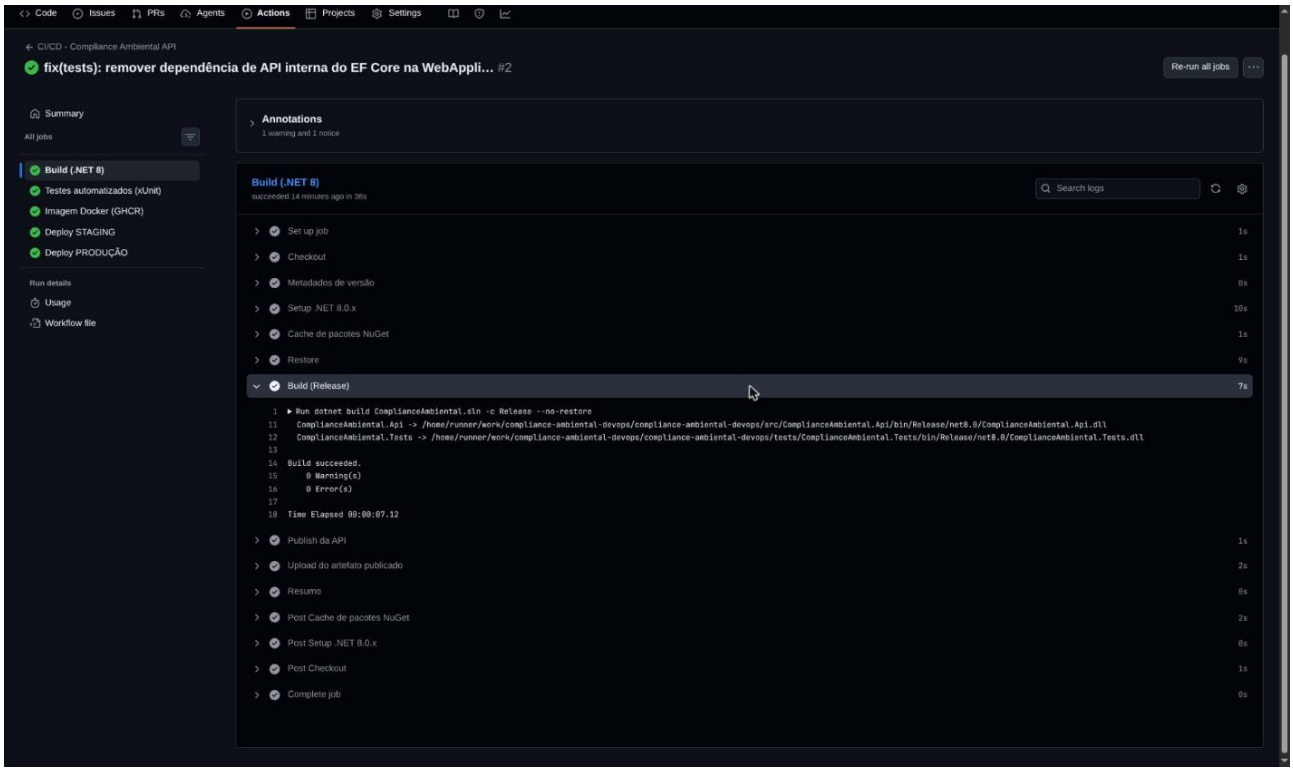


Figura 2 — Job build: dotnet build -c Release → "Build succeeded. 0 Warning(s) 0 Error(s)".

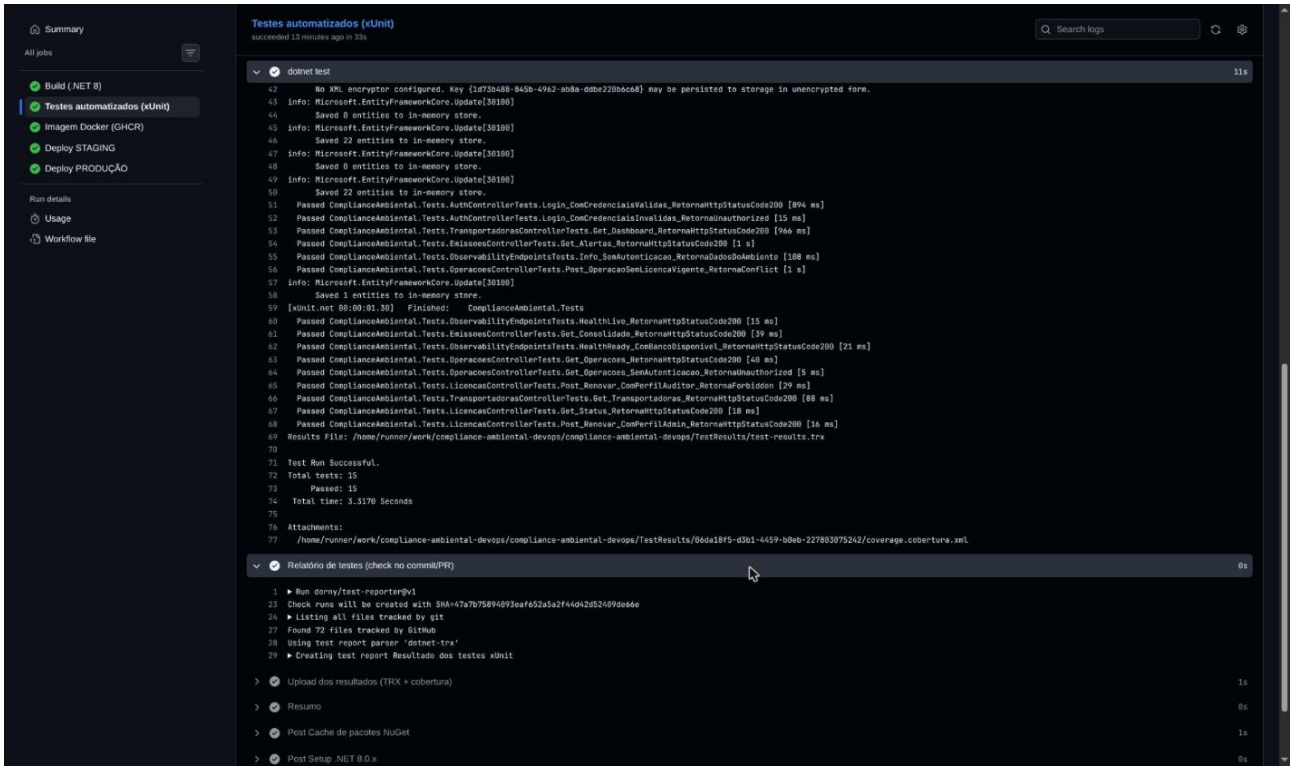


Figura 3 — Job test: 15 testes xUnit listados individualmente — "Test Run Successful. Total tests: 15, Passed: 15"; TRX e cobertura (cobertura.xml) gerados.

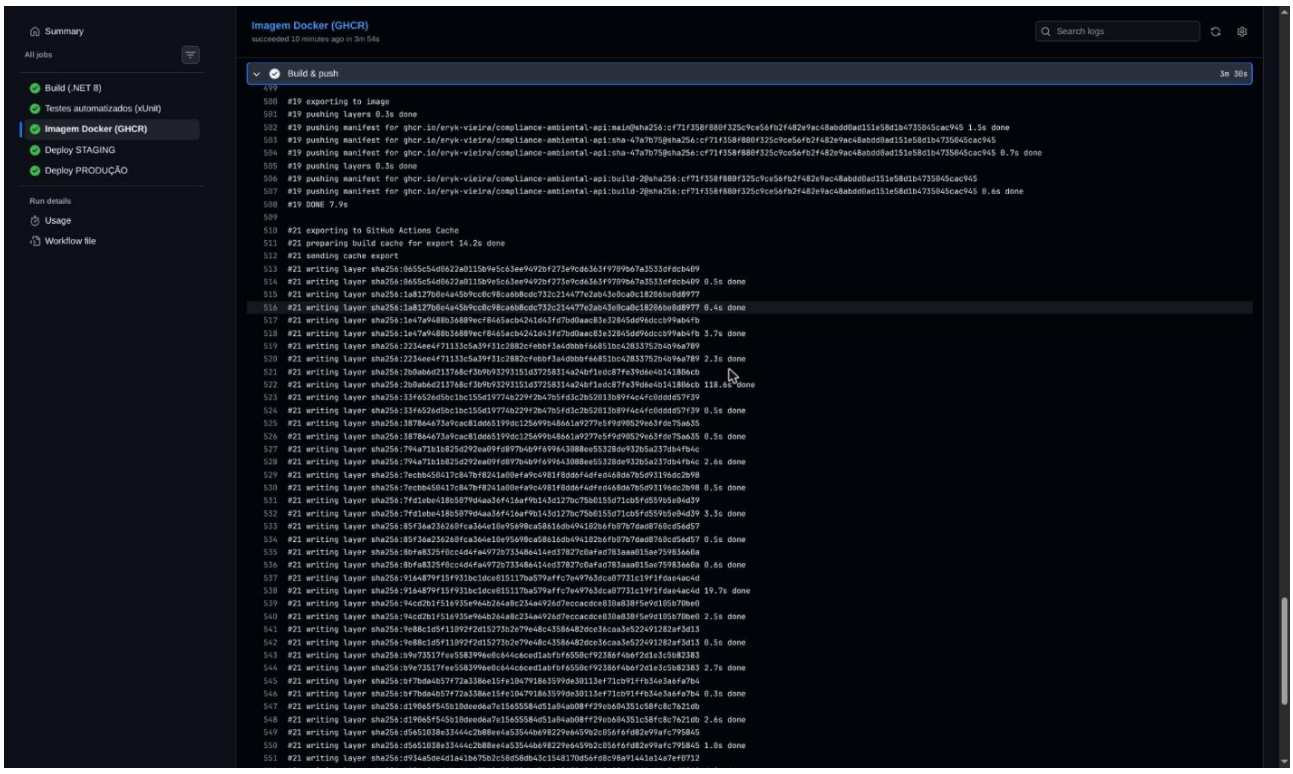


Figura 4 — Job docker: push dos manifests para ghcr.io/eryk-vieira/compliance-ambiental-api com as tags main, sha-47a7b75 e build-2, seguido da exportação do cache de camadas.

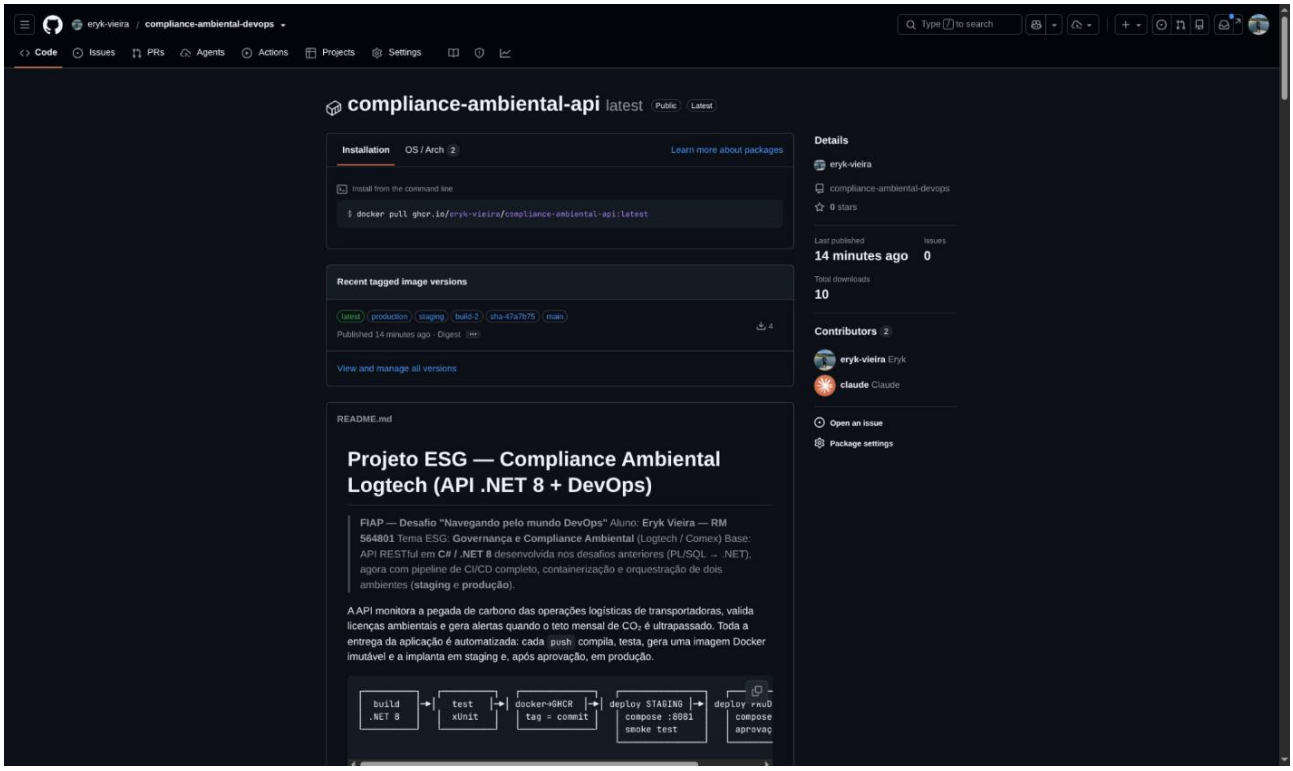


Figura 5 — Pacote público no GitHub Container Registry com as tags latest, production, staging, build-2, sha-47a7b75 e main (as tags de ambiente foram promovidas pelos jobs de deploy).

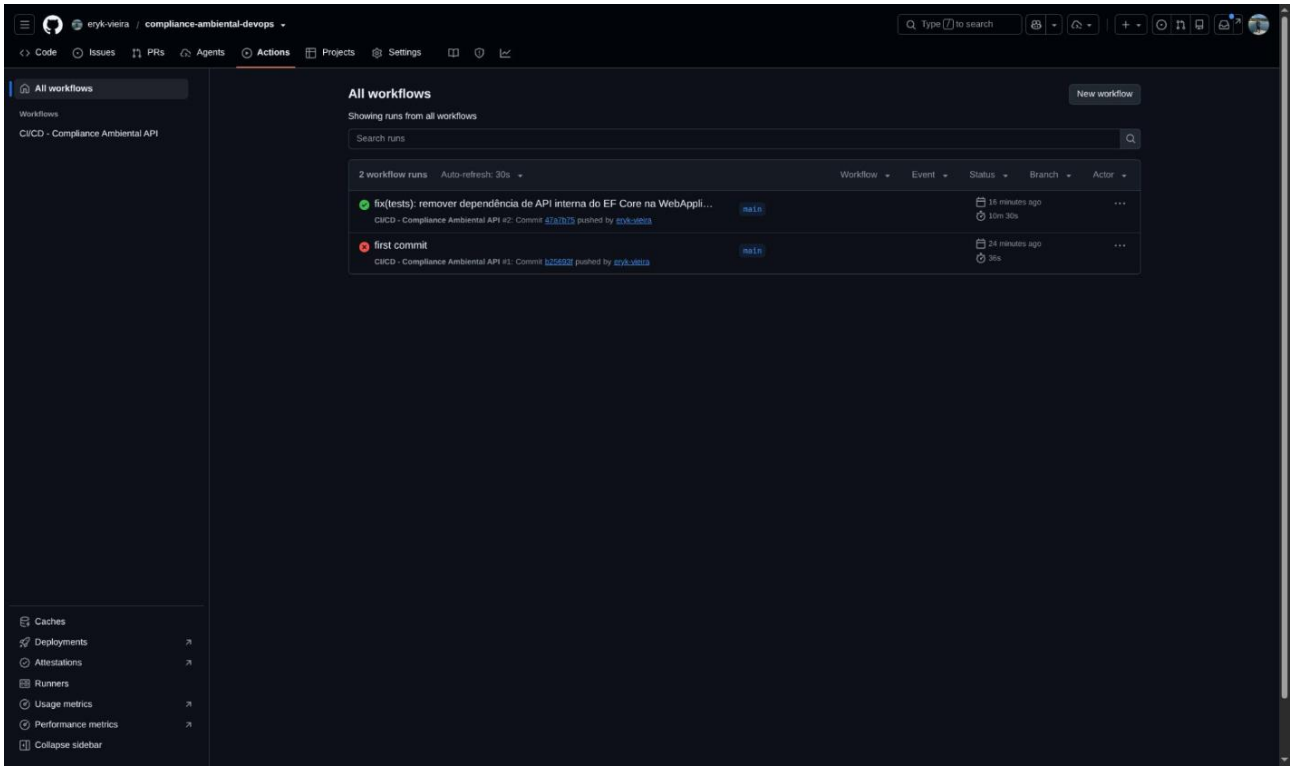


Figura 6 — Histórico de execuções: a 1ª execução falhou no build (ver seção 6) e a 2ª, após a correção, concluiu todo o fluxo.

5. Prints dos ambientes staging e produção funcionando

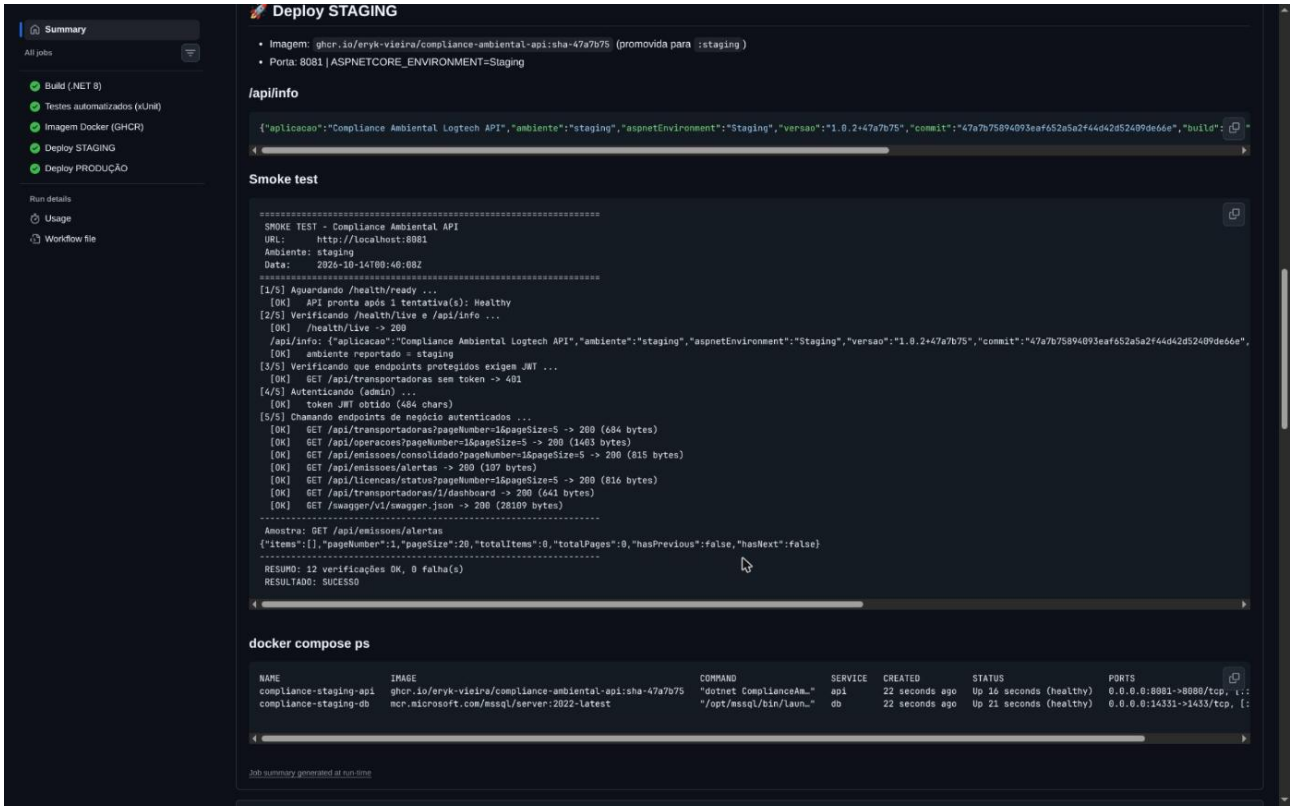


Figura 7 — STAGING: imagem sha-47a7b75 promovida para :staging; /api/info responde "ambiente":"staging" (porta 8081, ASPNETCORE_ENVIRONMENT=Staging); smoke test com 12 verificações OK; compose ps com api e db healthy.

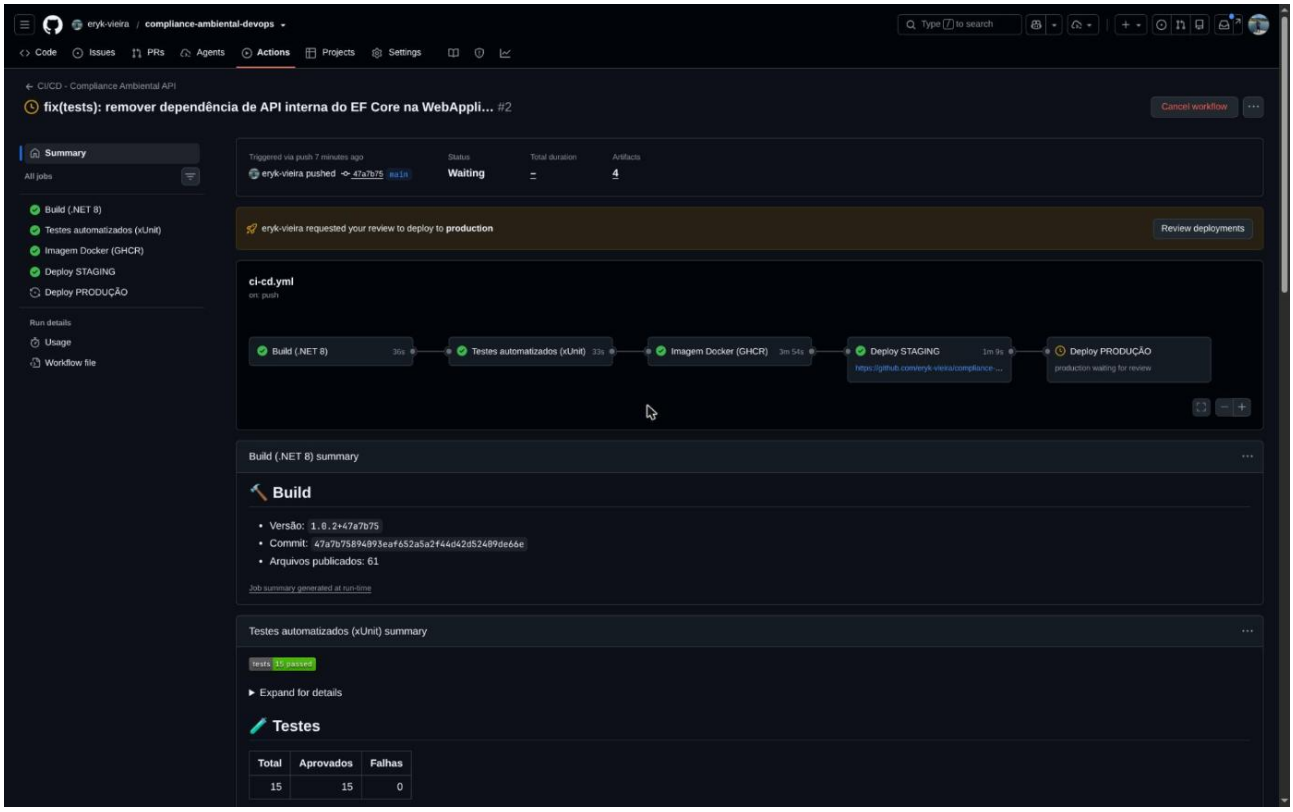


Figura 8 — Pipeline pausado após staging: "production waiting for review" — o environment production exige aprovação manual do revisor.

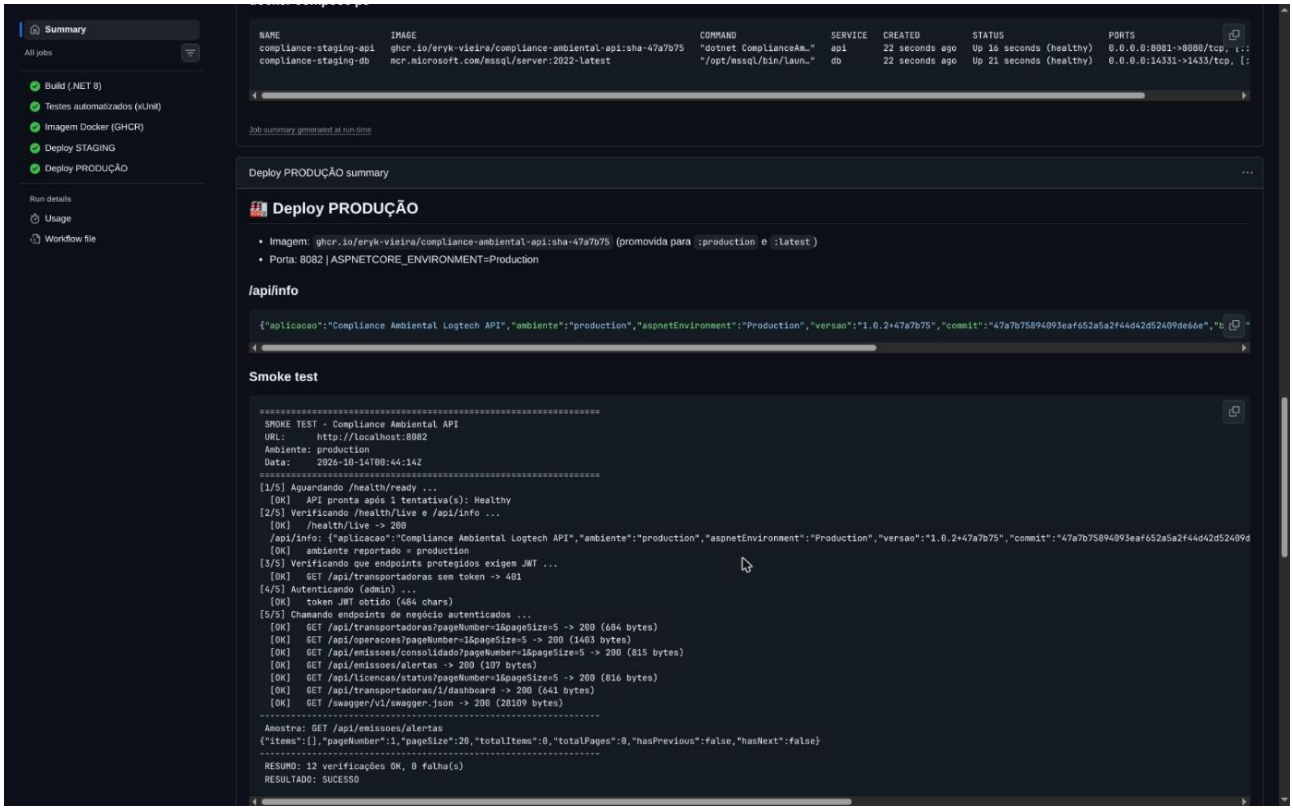


Figura 9 — PRODUÇÃO: imagem promovida para :production e :latest; /api/info responde "ambiente":"production" (porta 8082, ASPNETCORE_ENVIRONMENT=Production); smoke test 12/12 OK.

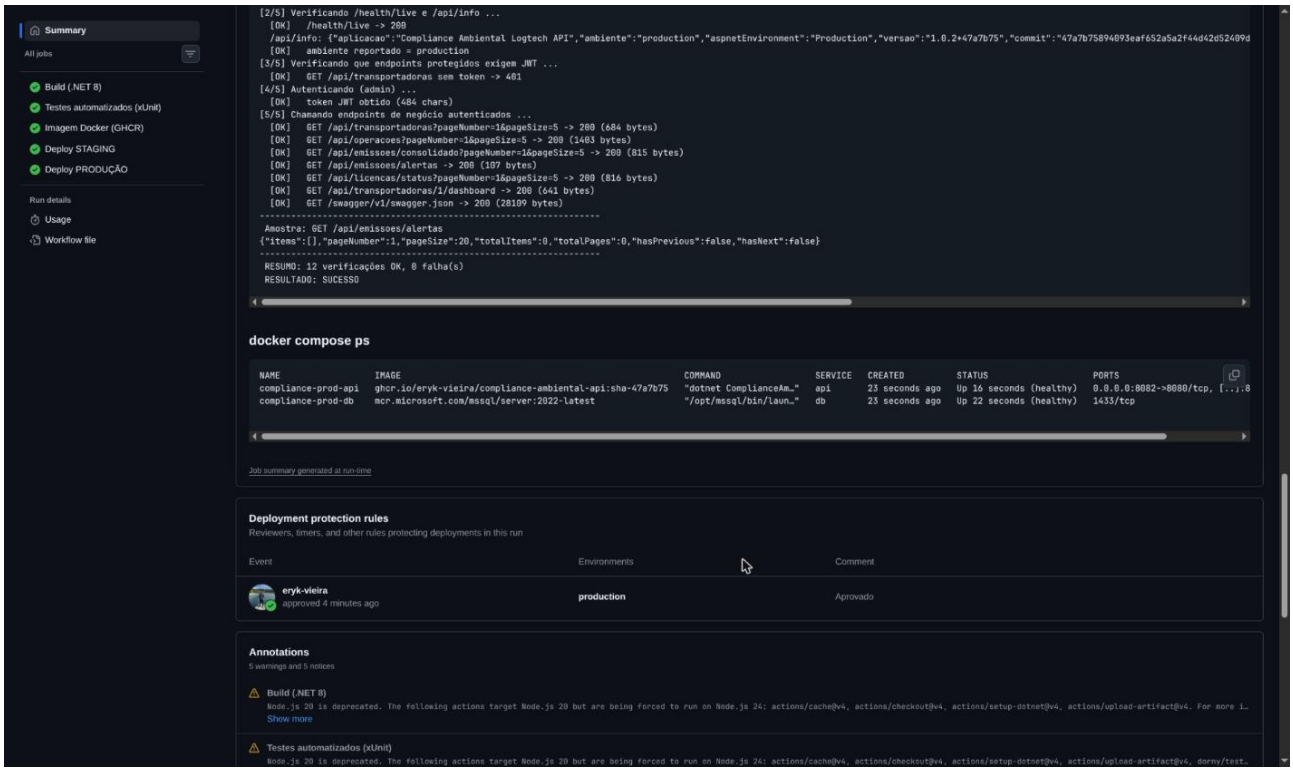


Figura 10 — Containers de produção healthy (banco com porta 1433 apenas interna) e registro da aprovação: "eryk-vieira approved production — Aprovado".

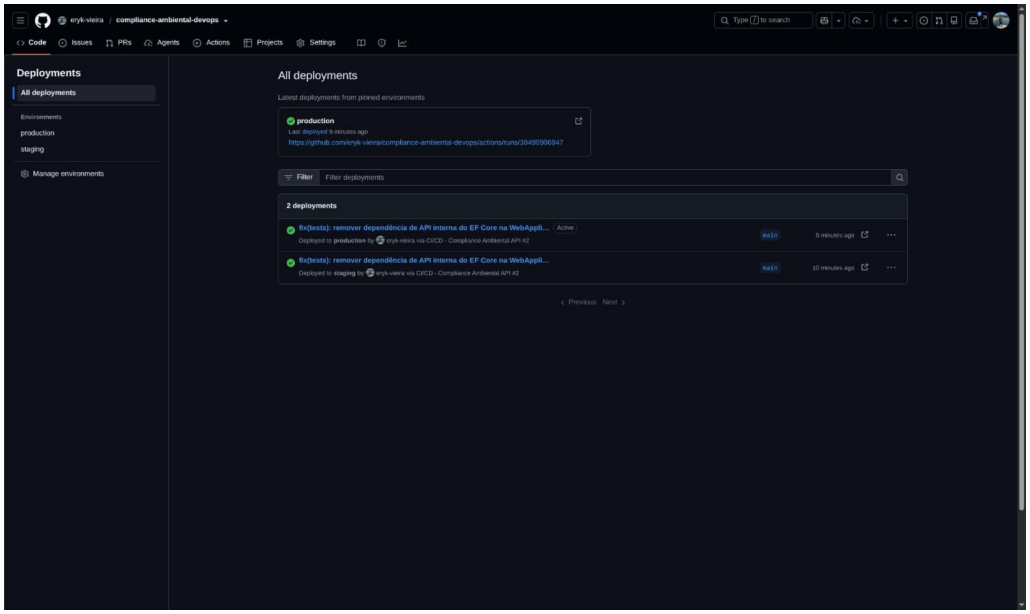


Figura 11 — Aba Deployments do repositório: histórico com o deploy em staging e em produção (ativo) da mesma versão.

Resposta de /api/info em cada ambiente (extraída dos Job Summaries)

```
STAGING  (http://localhost:8081/api/info)
{"aplicacao":"Compliance Ambiental Logtech API","ambiente":"staging","aspnetEnvironment":"Staging",
 "versao":"1.0.2+47a7b75","commit":"47a7b75894093eaf652a5a2f44d42d52409de66e","build":"2",
 "deployEm":"2026-10-14T00:39:20Z","horaServidorUtc":"2026-10-14T00:40:09Z","host":"1fd622af0552"}

PRODUÇÃO (http://localhost:8082/api/info)
{"aplicacao":"Compliance Ambiental Logtech API","ambiente":"production","aspnetEnvironment":"Production",
 "versao":"1.0.2+47a7b75","commit":"47a7b75894093eaf652a5a2f44d42d52409de66e","build":"2",
 "deployEm":"2026-10-14T00:43:35Z","horaServidorUtc":"2026-10-14T00:44:15Z","host":"82c05be2018b"}
```

Verificações do smoke test (idênticas nos dois ambientes)

#	Verificação	Resultado
1	GET /health/ready (API + banco)	Healthy na 1ª tentativa
2	GET /health/live	200
3	GET /api/info reporta o ambiente esperado	staging / production
4	GET /api/transportadoras sem token	401 (JWT obrigatório)
5	POST /api/auth/login (admin)	token JWT de 484 chars
6-12	GET transportadoras, operacoes, emissoes/consolidado, emissoes/alertas, licencas/status, transportadoras/1/dashboard, swagger.json (autenticados)	todos 200
	Resumo	**12 verificações OK, 0 falhas — SUCESSO**

6. Desafios encontrados e como foram resolvidos

Desafio	Como foi resolvido
Falha de compilação na 1ª execução do pipeline. O projeto de testes referenciava o tipo IDbContextOptionsConfiguration<T>, que é API interna do EF Core 8 (namespace Microsoft.EntityFrameworkCore.Internal); localmente compilava, mas no runner limpo o build falhou com CS0246.	O registro é localizado por reflexão pelo nome do tipo genérico (sem referenciar o namespace interno) e removido da coleção de serviços. O commit de correção disparou a 2ª execução, que passou integralmente. Ficou a lição: o CI em ambiente limpo é o que garante reprodutibilidade.
Provar que staging e produção são ambientes diferentes rodando a mesma imagem.	Endpoint /api/info alimentado por variáveis de ambiente (APP_ENVIRONMENT, APP_VERSION, APP_COMMIT_SHA, APP_BUILD_NUMBER, APP_DEPLOYED_AT) e smoke test que falha se o ambiente reportado não for o esperado. Compose files separados com portas, redes, volumes e bancos próprios.
SQL Server dentro do runner do GitHub (memória, tempo de inicialização, API subindo antes do banco).	MSSQL_MEMORY_LIMIT_MB=2048, healthcheck com sqlcmd, depends_on: condition: service_healthy, EnableRetryOnFailure no EF Core e compose up --wait --wait-timeout 300 — o deploy só prossegue com os dois containers healthy.
Healthcheck da API: a imagem base aspnet:8.0 não tem curl e roda como usuário não-root.	Instalação mínima de curl no estágio de runtime e HEALTHCHECK apontando para /health/ready (que verifica o banco); USER app mantido.
Segredos e configuração por ambiente sem versioná-los nem copiá-los para a imagem.	Apenas .env.*.example versionados; .env reais ignorados pelo git e pelo .dockerignore; no pipeline o .env.<ambiente> é gerado a partir dos Secrets/Variables do environment (com valores padrão para a avaliação).
Promover a mesma imagem sem rebuild (evitar que produção receba um binário diferente do testado).	Tag imutável sha-<commit> gerada uma vez; jobs de deploy apenas re-tagueiam no registro com docker buildx imagetools create (:staging, :production, :latest).
Aprovação manual de produção e rastreabilidade do deploy.	GitHub Environment production com required reviewers; o pipeline pausa ("waiting for review") e registra quem aprovou e quando; aba Deployments mostra o histórico por ambiente.
Evidências reproduzíveis sem depender de prints manuais.	scripts/collect-evidence.sh grava compose ps, imagens, redes, volumes, labels, logs e respostas da API; o pipeline publica tudo como artefatos (evidence-staging/production-<sha>) e no Job Summary.

7. Checklist de entrega

Item	OK	Onde conferir
Projeto compactado em .ZIP com estrutura organizada	☑	compliance-ambiental-devops.zip (Dockerfile, docker-compose*.yml, src/, tests/, scripts/, docs/, README.md, .github/workflows/)
Dockerfile funcional	☑	Dockerfile — imagem publicada em ghcr.io/eryk-vieira/compliance-ambiental-api
docker-compose.yml ou arquivos Kubernetes	☑	docker-compose.yml, docker-compose.staging.yml, docker-compose.production.yml
Pipeline com etapas de build, teste e deploy	☑	.github/workflows/ci-cd.yml — run #2 Success (5 jobs)
README.md com instruções e prints	☑	README.md (11 prints em docs/prints/)
Documentação técnica com evidências (PDF ou PPT)	☑	docs/Documentacao_Tecnica_DevOps.pdf (este documento)
Deploy realizado nos ambientes staging e produção	☑	Jobs deploy-staging e deploy-production; aba Deployments do repositório

Links

Repositório: https://github.com/eryk-vieira/compliance-ambiental-devops
Execução do pipeline: https://github.com/eryk-vieira/compliance-ambiental-devops/actions/runs/38490906947
Imagem Docker:
https://github.com/eryk-vieira/compliance-ambiental-devops/pkgs/container/compliance-ambiental-api
Deployments: https://github.com/eryk-vieira/compliance-ambiental-devops/deployments